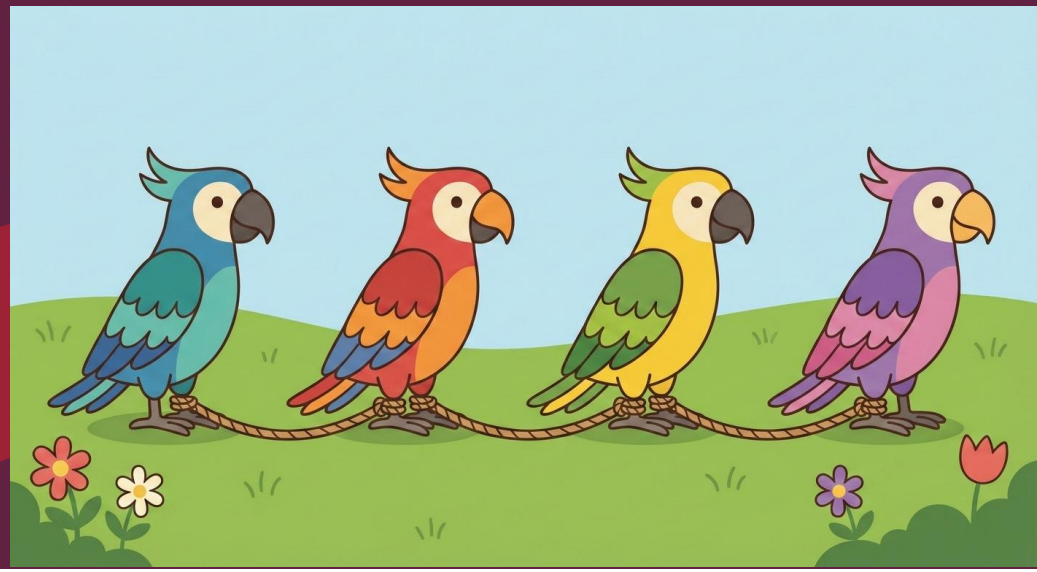
Agentic AI with LangChain and LangGraph



Presentation 3: Intro to LangChain
Primary Instructor: Michael Z
RA: YH
Date: 2026.Jan-Feb

Sobey School of Business
Saint Mary's University
MBAN

Saint Mary's University

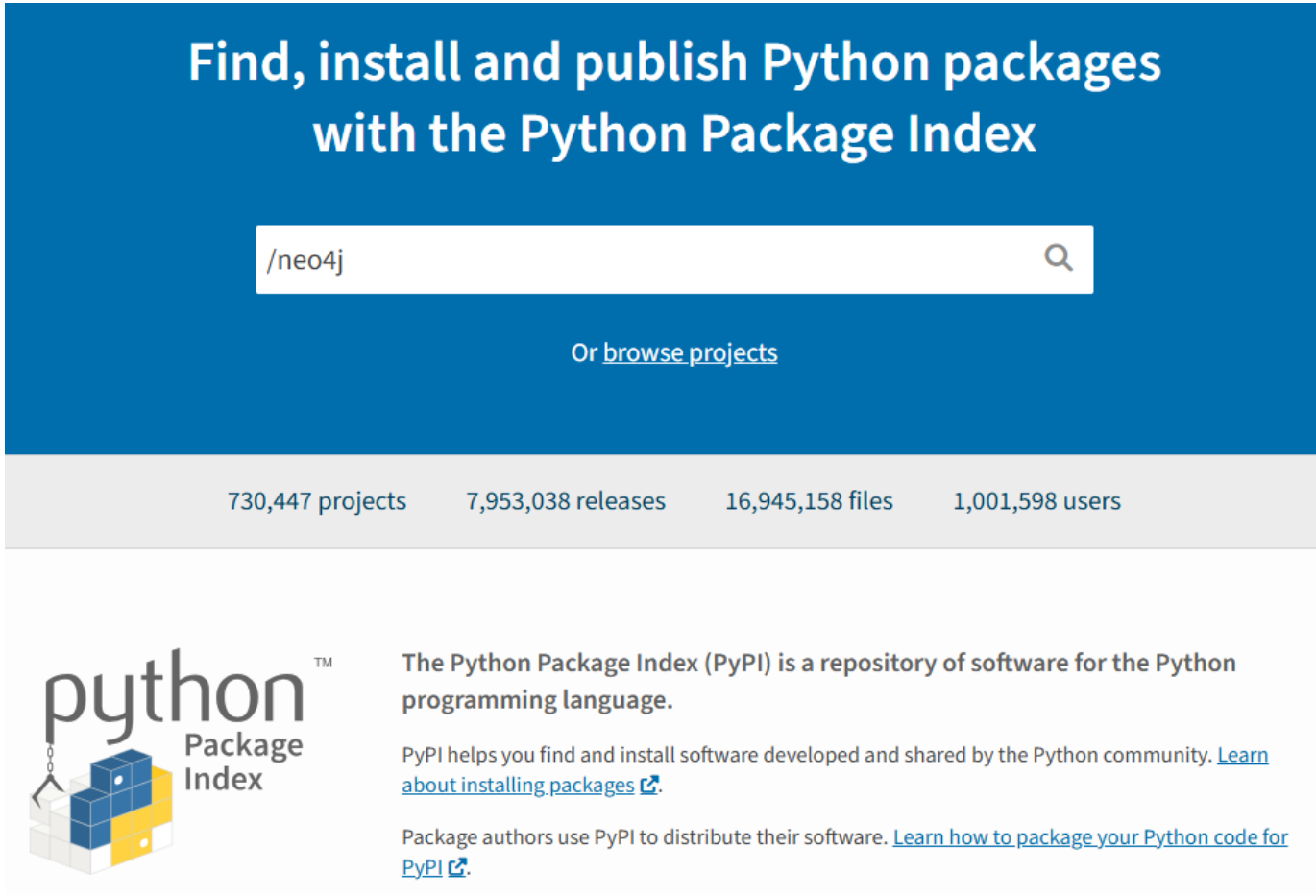
0. Common Sense & A Quick Review of QuickStart

0. A Quick Review of QuickStart: What? I need to install packages?

You made a Python package that help you make use of AI to make AI Agents whenever you want. Let's call it ai-agents-factory. And obviously, native Python doesn't have this functionality, and you also want others to use. Then you build the wheel and publish it.

This Python package would be available in PyPi, which is maintained by Python Software Foundation, the organization behind Python language.

Of course, if what you publish contains virus, it would be removed and nobody can search it through pip or any other python package managers.



0. A Quick Review of QuickStart: What? I need to install packages?

You keep updating it and publish multiple versions (releases management). One day, you found the following feature from Python 3.10 useful then you decide to use it in newer version of your Python package. Let's say 2.0.

> *Changed in version 3.10:* Unions can now be written as `X | Y`. See union type expressions.

Then, regardless of what Python package manager you are using, uv or native pip, as long as you are using Python <3.10, ai-agents-factory 2.0 would **NEVER** be indexed and downloaded. This is why some of you can not download new langchain (1.2.x+). If you are using Python <3.11, old langchain would be indexed and downloaded.

Old langchain is not the focus of LangChain Inc but it's still being maintained.

Btw we are learning new langchain, as stated in Presentation 1.

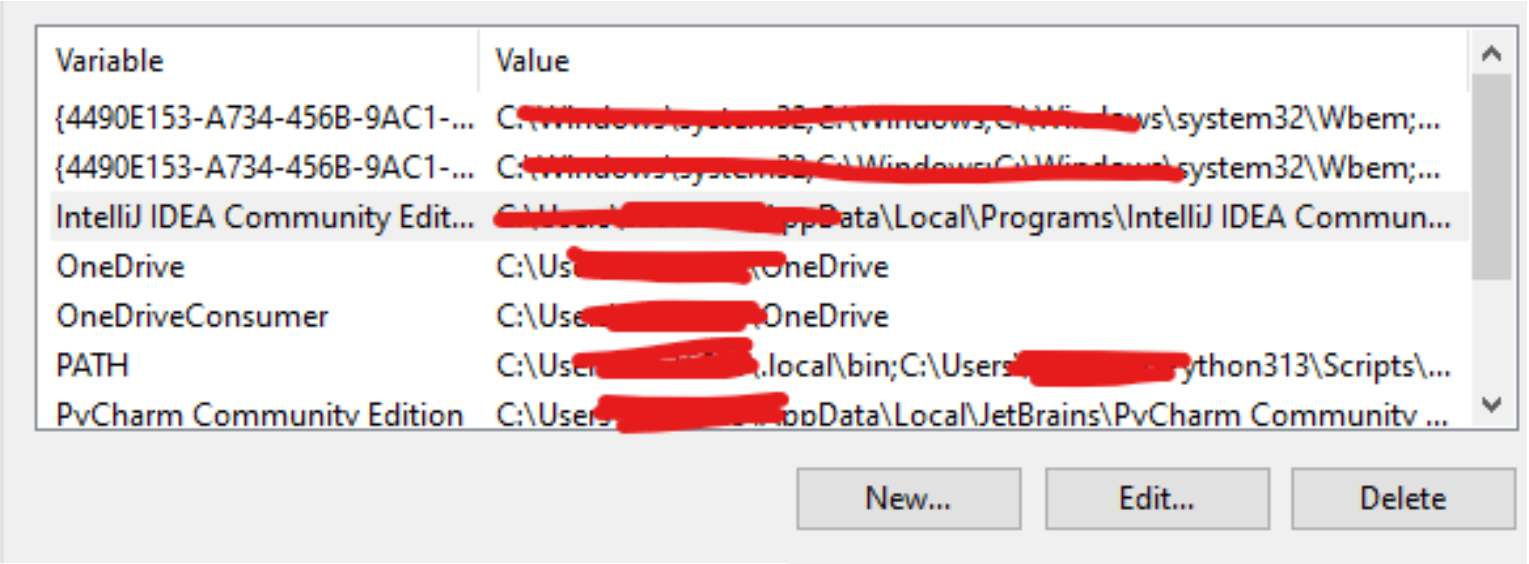
One of the reasons that I don't like colab is because it has a lot of packages pre-installed and you might never realize you need to install additional packages.

0. A Quick Review of QuickStart: Environment Variables

I am not teaching computer literacy but just to let you know that in windows OS, environment variables are stored like key-value pairs. The OPENAI_API_KEY is the variable, the sk-……. in the email I sent you is the value.

You could add OPENAI_API_KEY in it for sure but to make changes take effect, you need to restart your machine, which is a little bit inconvenient. If you do that, it could be really convenient when programming at some point because under the hood, openai, the python package would try to find an environment variable called “OPENAI_API_KEY” and retrieve its value. But when you need to update your key, you might have to restart your machine.

This is why I told you to use .env file, which is more flexible. Updating API key doesn’t need to restart anything.



0. A Quick Review of QuickStart: API vs API key

One day, Michael deployed a home-made LLM in a server with public IP address. He wants to sell this LLM. So he made a Python package and named it “michael-llm”.

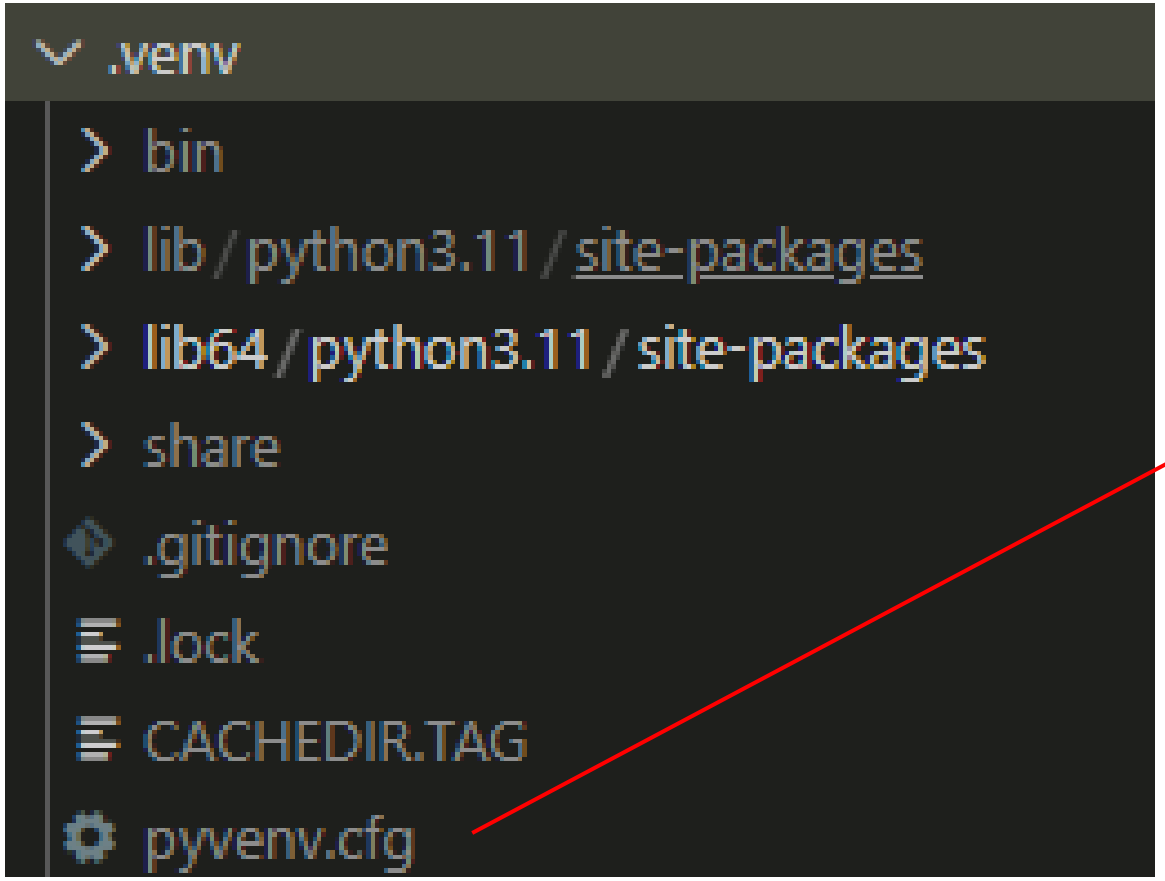
Anyone who downloaded this package meets the technical prereq to use michael’s LLM.

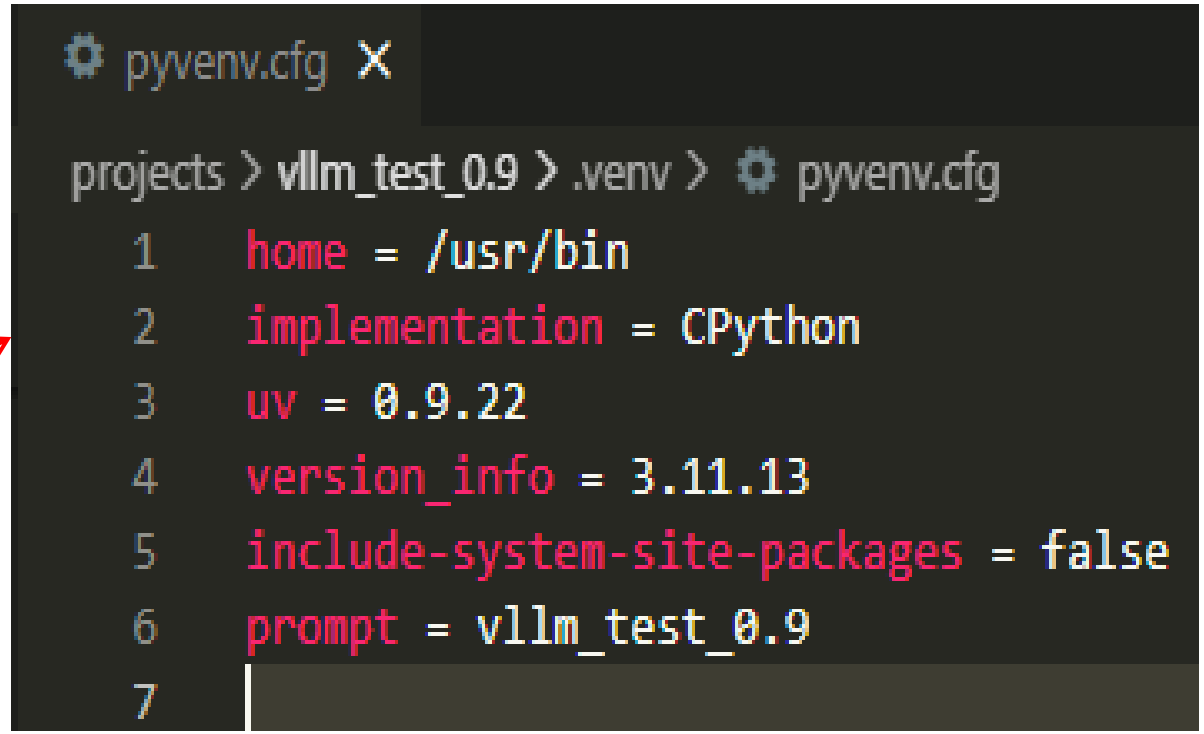
But Michael doesn’t want everyone to use. Then he sent some keys to those who paid for this service so that only the key holders could access his LLM.

We call the key “API key”. The functions you use to access michael’s LLM are called “API”.

0. A Quick Review of QuickStart: Virtual Environment

Click the .venv folder created in linux system (Rocky Linux Enterprise)…….

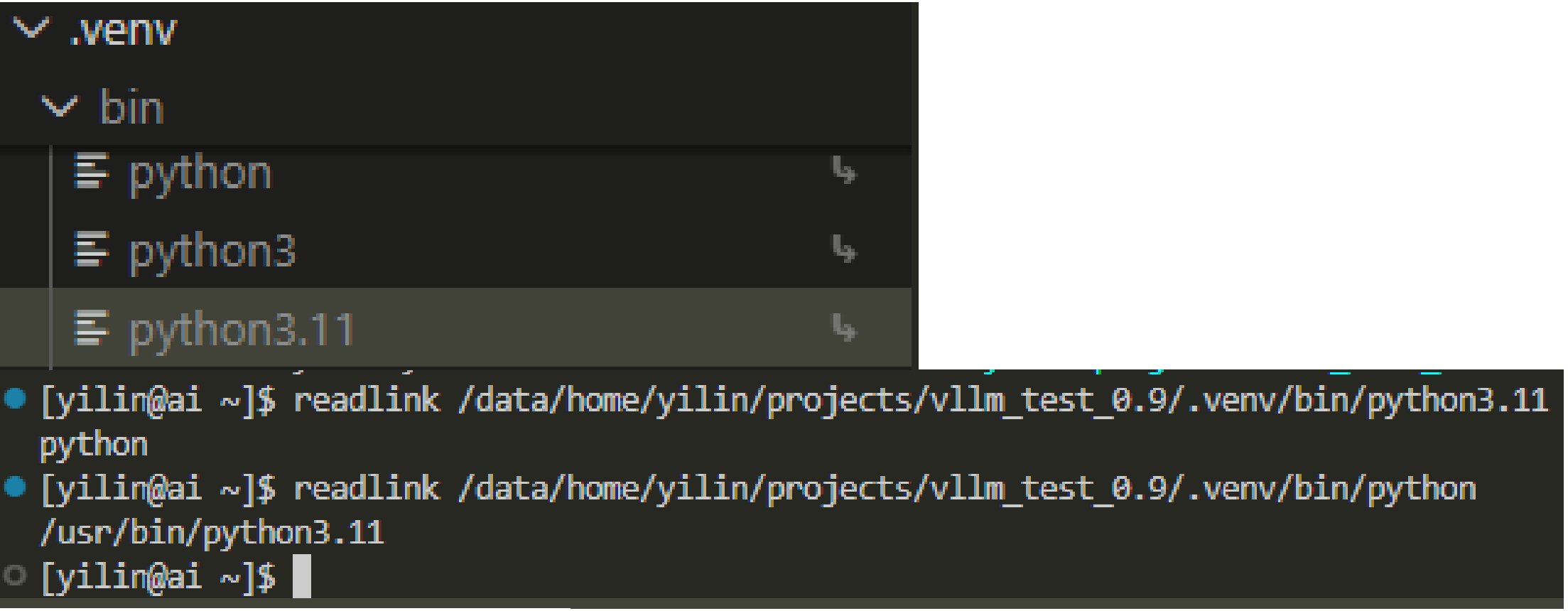




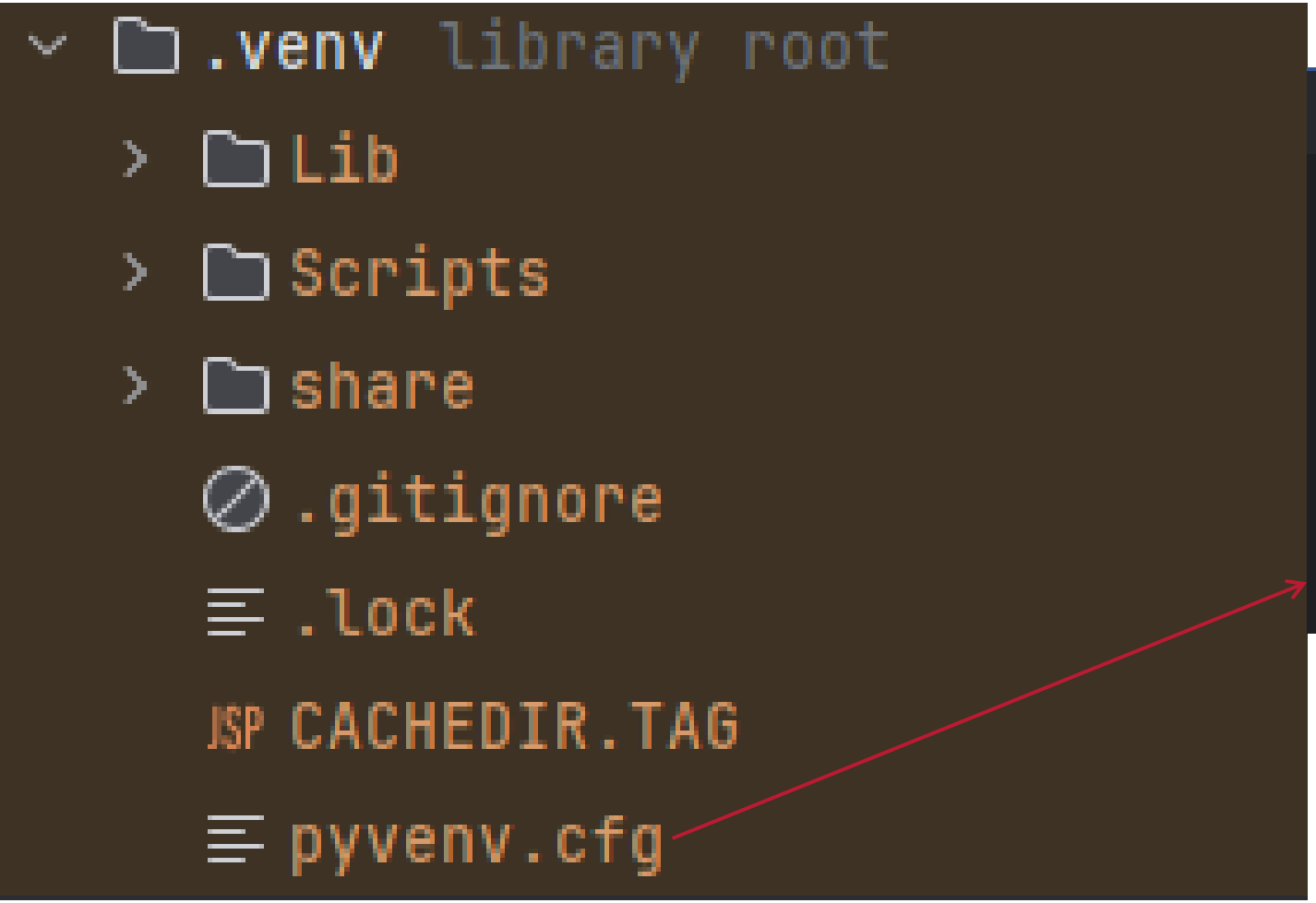
0. A Quick Review of QuickStart: Symbolic Link

Inside the .venv/bin, you could find three files (symbolic links). They are not actual Python interpreters but all point to actual Python interpreter, which is located in usr/bin/python3.11.

Use readlink to see what these files are actually pointing to……..



0. A Quick Review of QuickStart: .venv in Windows OS




```
home = C:\Users\[redacted]\Python313
implementation = CPython
uv = 0.8.3
version_info = 3.13.5
include-system-site-packages = false
prompt = pythonproject
```

0. A Quick Review of QuickStart: Global Python Interpreter in Windows

DLLs	6/24/2025 12:52 PM	File folder	
Doc	6/24/2025 12:52 PM	File folder	
include	7/25/2025 5:59 PM	File folder	
Lib	6/24/2025 12:52 PM	File folder	
libs	6/24/2025 12:52 PM	File folder	
Scripts	10/4/2025 8:09 PM	File folder	
share	7/13/2025 1:48 PM	File folder	
tcl	6/24/2025 12:52 PM	File folder	
LICENSE.txt	6/11/2025 6:11 PM	Text Document	34 KB
NEWS.txt	6/11/2025 6:22 PM	Text Document	1,953 KB
python.exe	6/11/2025 6:10 PM	Application	104 KB
python3.dll	6/11/2025 6:10 PM	Application exten...	71 KB
python313.dll	6/11/2025 6:10 PM	Application exten...	5,968 KB
pythonw.exe	6/11/2025 6:10 PM	Application	102 KB
vcruntime140.dll	6/11/2025 6:11 PM	Application exten...	118 KB
vcruntime140_1.dll	6/11/2025 6:11 PM	Application exten...	49 KB

0. A Quick Review of QuickStart: QuickStart

```python
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"

agent = create_agent(
    model="claude-sonnet-4-5-20250929",
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)

# Run the agent
agent.invoke(
    {"messages": [{"role": "user", "content": "what is the weather in sf"}]}
)
```

Recipe:

1. Install uv
2. Find an empty folder and **pin** a version of python to it (let's say 3.11). Then initialize it (uv init) and install langchain, langchain-openai, python-dotenv using Python 3.11 or higher (uv add …….).
3. Replace "claude-….." with ChatOpenAI(model = "….", api_key = os.getenv("OPENAI_API_KEY"))
4. Add load_dotenv() after imports.
5. Create a variable to hold what's returned by invoke function and then print it out.
6. Create .env file then put OPENAI_API_KEY=sk…… in it.
7. In terminal, uv run ….. .py

uv python pin

Pin to a specific Python version.

Writes the pinned Python version to a `.python-version` file, which is used by other uv commands to determine the required Python version.

If no version is provided, uv will look for an existing `.python-version` file and display the currently pinned version. If no `.python-version` file is found, uv will exit with an error.

See `uv help python` to view supported request formats.

Usage

```
uv python pin [OPTIONS] [REQUEST]
```

https://docs.astral.sh/uv/reference/cli/#uv-python-pin

0. A Quick Review of QuickStart: What's Returned?

```
[yilin@ai quickstart_of_lc]$ uv run main.py
{'messages': [HumanMessage(content='what is the weather in sf', additional_kwargs={}, response_metadata={}, id='6f05e083-9937-46ff-a603-7af7e4014a21'), AIMessage(content='', additional_kwargs={'refusal': None}, response_me
tadata={'token_usage': {'completion_tokens': 15, 'prompt_tokens': 56, 'total_tokens': 71, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens
': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-4o-2024-08-06', 'system_fingerprint': 'fp_deacdd5f6f', 'id': 'chatcmpl-D20UQAoez16QSEtMaay6jpFFpsMOO'
, 'service_tier': 'default', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019bf6b9-7861-71b1-8aa8-a05f39f646ed-0', tool_calls=[{'name': 'get_weather', 'args': {'city': 'San Francisco'}, 'id': 'call_2TN2ceM
70Uv2hMp2IL639xyH', 'type': 'tool_call'}], invalid_tool_calls=[], usage_metadata={'input_tokens': 56, 'output_tokens': 15, 'total_tokens': 71, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {
'audio': 0, 'reasoning': 0}}), ToolMessage(content="It's always sunny in San Francisco!", name='get_weather', id='89e729b9-7b30-4c2e-8ee9-b5dd32e8f531', tool_call_id='call_2TN2ceM70Uv2hMp2IL639xyH'), AIMessage(content='Cur
rently, the weather in San Francisco is sunny. 🌞, additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 14, 'prompt_tokens': 86, 'total_tokens': 100, 'completion_tokens_details': {'a
ccepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-4
o-2024-08-06', 'system_fingerprint': 'fp_deacdd5f6f', 'id': 'chatcmpl-D20UR4kEVbKNE7mLBJfjZl8YXnyMG', 'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019bf6b9-80b4-7321-9920-f6e89887365e-
0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 86, 'output_tokens': 14, 'total_tokens': 100, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning'
: 0}})]}
```

0. A Quick Review of QuickStart: FYI

```python
import os
from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI
load_dotenv()
def get_weather(city: str) -> str:
    """Get weather for a given city."""
    return f"It's always sunny in {city}!"
agent = create_agent(
    model=ChatOpenAI(model = "gpt-4o", api_key = os.getenv("OPENAI_API_KEY")),
    tools=[get_weather],
    system_prompt="You are a helpful assistant",
)
# Run the agent
msg = agent.invoke(
    {"messages": [{"role": "user", "content": "what is the weather in sf"}]}
)
print(msg)
```

(parameter) city: str

```
[yilin@ai ~]$ cd projects/quickstart_of_lc
[yilin@ai quickstart_of_lc]$ uv python pin 3.11
Pinned `.python-version` to `3.11`
[yilin@ai quickstart_of_lc]$ uv init
Initialized project `quickstart-of-lc`
[yilin@ai quickstart_of_lc]$ ls -a
. .. .git .gitignore main.py pyproject.toml .python-version README.md
[yilin@ai quickstart_of_lc]$ uv add langchain langchain-openai python-dotenv
Using CPython 3.11.13 interpreter at: /usr/bin/python3.11
Creating virtual environment at: .venv
Resolved 43 packages in 785ms
Prepared 27 packages in 452ms
Installed 41 packages in 156ms
 ……..
[yilin@ai quickstart_of_lc]$ uv run main.py
{'messages': [HumanMessage(content='what is the weather in sf', additional_kwargs={},
response_metadata={}, id='6f05e083-9937-46ff-a603-7af7e4014a21'), AIMessage(content='',
additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 15,
'prompt_tokens': 56, 'total_tokens': 71, 'completion_tokens_details': {'accepted_prediction_tokens':
0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details':
{'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-4o-2024-08-
06', 'system_fingerprint': 'fp_deacdd5f6f', 'id': 'chatcmpl-D20UQAoez16QSEtMaay6jpFFpsMOO',
'service_tier': 'default', 'finish_reason': 'tool_calls', 'logprobs': None}, id='lc_run--019bf6b9-7861-
71b1-8aa8-a05f39f646ed-0', tool_calls=[{'name': 'get_weather', 'args': {'city': 'San Francisco'}, 'id':
'call_2TN2ceM70Uv2hMp2IL639xyH', 'type': 'tool_call'}], invalid_tool_calls=[],
usage_metadata={'input_tokens': 56, 'output_tokens': 15, 'total_tokens': 71, 'input_token_details':
{'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}}),
ToolMessage(content="It's always sunny in San Francisco!", name='get_weather', id='89e729b9-
7b30-4c2e-8ee9-b5dd32e8f531', tool_call_id='call_2TN2ceM70Uv2hMp2IL639xyH'),
AIMessage(content='Currently, the weather in San Francisco is sunny. 🌞',
additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 14,
'prompt_tokens': 86, 'total_tokens': 100, 'completion_tokens_details': {'accepted_prediction_tokens':
0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details':
{'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-4o-2024-08-
06', 'system_fingerprint': 'fp_deacdd5f6f', 'id': 'chatcmpl-D20UR4kEVbKNE7mLBJfjZI8YXnyMG', 
'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019bf6b9-80b4-7321-
9920-f6e89887365e-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 86,
'output_tokens': 14, 'total_tokens': 100, 'input_token_details': {'audio': 0, 'cache_read': 0},
'output_token_details': {'audio': 0, 'reasoning': 0}})]}
```

1. Core Components of LangChain (without Middleware)

1. Core Components of LangChain (without Middleware)

Since you have done quickstart (if you haven't that's okay), we can start to play with LangChain right now!

Contents relating to Middleware would be skipped for now because they are a little bit hard for you right now.

We would go over some (not all) core components, as listed in LangChain website. If you are comfortable to learn more on your own, feel free to go ahead and don't need to follow my pace!

Hope you have fun!

Core components

Agents

Models

Messages

Tools

Short-term memory

Streaming

Structured output

*If you want to play-with-LangChain-as-I-talk:

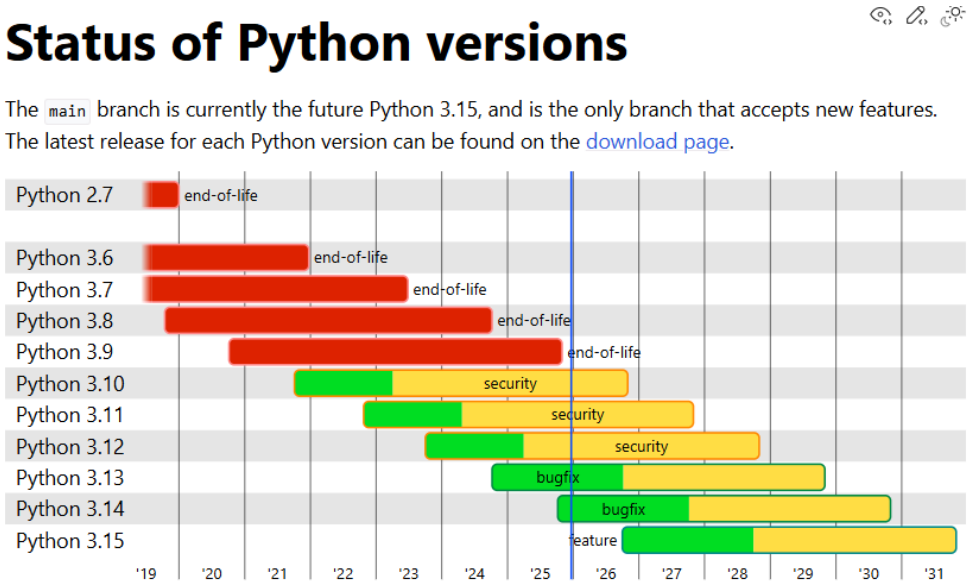
Assuming you all installed uv, then make a folder in a location you like (not C drive in windows btw). LangChain suggests to use Python 3.10 or higher at the time I was writing this slide.

A specific python version would become “end-of-life” after a period of time so please be advised to upgrade to suitable python version regularly.

Then pin a version you like, init it and see what you have
(I am using linux to do the following and so please adapt your command lines to the OS you are using.)

And then uv add langchain (and whatever you might need).
If you are happy with it, uv run main.py (or whatever you like)

Read uv docs to learn more.




```
[yilin@ai mban]$ uv python pin 3.12
Pinned `.python-version` to `3.12`
[yilin@ai mban]$ uv init
Initialized project `mban`
[yilin@ai mban]$ ls
main.py  pyproject.toml  README.md
[yilin@ai mban]$ ls -a
.  ..  .git  .gitignore  main.py  pyproject.toml  .python-version  README.md
[yilin@ai mban]$ nano .python-version
[yilin@ai mban]$
```

1.1 Core Components of LangChain (without Middleware): Models

I think I would switch between slides and LangChain website of docs frequently since they already wrote docs for us.

But I would select some topics that you need to know for today.

We would talk about Models first.

1. Model Initialization
2. Invocation
3. Tools calling
4. Structured output
5. Runtime-Configurable models

As you can see that model itself, without being wrapped as ReAct agent, could do a lot of things.

1.1.1 Core Components of LangChain (without Middleware): Models: Init, Invocation and Runtime-Configurable Models

Recall:

model `string` `required`

The name or identifier of the specific model you want to use with a provider. You can also specify both the model and its provider in a single argument using the ':' format, for example, 'openai:o1'.

api_key `string`

The key required for authenticating with the model's provider. This is usually issued when you sign up for access to the model. Often accessed by setting an environment variable.

temperature `number`

Controls the randomness of the model's output. A higher number makes responses more creative; lower ones make them more deterministic.

max_tokens `number`

Limits the total number of tokens in the response, effectively controlling how long the output can be.

timeout `number`

The maximum time (in seconds) to wait for a response from the model before canceling the request.

max_retries `number`

The maximum number of attempts the system will make to resend a request if it fails due to issues like network timeouts or rate limits.

After you initialize a LLM successfully, you can send something as input to this LLM, which is called "invocation".

You can invoke the LLM with multiple independent requests in parallel as well, which is called "batch".

You could make invocation/batch interactive to improve user experience, which is called "stream".

If you want to decide what kind of model to invoke, what temperature to use and so on at invocation, rather than initialization, you can use parameter configurable_fields and specify what could be determined dynamically.

1.1.1 Core Components of LangChain (without Middleware): Models: Init, Invocation and Runtime-Configurable Models

There are 4 types of Messages in LangChain but most of the time you would use 3 of them. SystemMessage is system prompt and HumanMessage is user prompt, which is what you typed in to LLM when playing with ChatGPT. AIMessage means LLM's output.

You could do it in several ways, dict or using their pre-defined classes. They serve the same purpose.
If you don't specify anything and just throw a sentence to invoke function, it would be considered as user prompt.
You might use AIMessage in invoke function sometimes for one-shot prompting or few-shot prompting (What's attached below is GPT-3 paper that tells you few-shot prompting impact. FYI).

[Submitted on 28 May 2020 (v1), last revised 22 Jul 2020 (this version, v4)]

Language Models are Few-Shot Learners

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, Dario Amodei

Recent work has demonstrated substantial gains on many NLP tasks and benchmarks by pre-training on a large corpus of text followed by fine-tuning on a specific task. While typically task-agnostic in architecture, this method still requires task-specific fine-tuning datasets of thousands or tens of thousands of examples. By contrast, humans can generally perform a new language task from only a few examples or from simple instructions - something which current NLP systems still largely struggle to do. Here we show that scaling up language models greatly improves task-agnostic, few-shot performance, sometimes even reaching competitiveness with prior state-of-the-art fine-tuning approaches. Specifically, we train GPT-3, an autoregressive language model with 175 billion parameters, 10x more than any previous non-sparse language model, and test its performance in the few-shot setting. For all tasks, GPT-3 is applied without any gradient updates or fine-tuning, with tasks and few-shot demonstrations specified purely via text interaction with the model. GPT-3 achieves strong performance on many NLP datasets, including translation, question-answering, and cloze tasks, as well as several tasks that require on-the-fly reasoning or domain adaptation, such as unscrambling words, using a novel word in a sentence, or performing 3-digit arithmetic. At the same time, we also identify some datasets where GPT-3's few-shot learning still struggles, as well as some datasets where GPT-3 faces methodological issues related to training on large web corpora. Finally, we find that GPT-3 can generate samples of news articles which human evaluators have difficulty distinguishing from articles written by humans. We discuss broader societal impacts of this finding and of GPT-3 in general.

1.1.2 Core Components of LangChain (without Middleware): Models: Tools Calling

A quick questions for you:

How is the capability of tools calling obtained? If you remember, you won't be confused by the following.

Tool execution loop

```
# Bind (potentially multiple) tools to the model
model_with_tools = model.bind_tools([get_weather])

# Step 1: Model generates tool calls
messages = [{"role": "user", "content": "What's the weather in Boston?"
ai_msg = model_with_tools.invoke(messages)
messages.append(ai_msg)

# Step 2: Execute tools and collect results
for tool_call in ai_msg.tool_calls:
    # Execute the tool with the generated arguments
    tool_result = get_weather.invoke(tool_call)
    messages.append(tool_result)

# Step 3: Pass results back to model for final response
final_response = model_with_tools.invoke(messages)
print(final_response.text)
# "The current weather in Boston is 72°F and sunny."
```

Define your own tools and have fun (easy tools I mean) !!!

Some vendors built their own tools (server-side tool. You are at client-side.) and you can use them if you like!

(Tool is just a function plus a decorator lol)

Invoke with server-side tool use

```
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-4.1-mini")

tool = {"type": "web_search"}
model_with_tools = model.bind_tools([tool])

response = model_with_tools.invoke("What was a positive news story from today
response.content_blocks
```

1.1.3 Core Components of LangChain (without Middleware): Models: Structured Output

"Structured generation is the future of LLMs."

Julien Chaumond, Cofounder & CTO, HuggingFace

Figure 1: From website of dottxt.ai, creator of outlines.

1.2.1 Core Components of LangChain (without Middleware): Agents: Tools Calling, Structured Output

What you would do with agents for Tools Calling and Structured Output is similar to what you did in “Models”, but something is automated for you, and something is a little bit different.

Two strategy for structured output: ToolStrategy and ProviderStrategy.

I would leave a question for you:

What are differences between model and agent? I won’t give you answer but would like to know what you think.

1.2.2 Core Components of LangChain (without Middleware): Agents: Short-Term Memory

Agent could remember some previous interactions through thread id (Thread here doesn't mean the thread in Operating System, just treat it as an identifier!!!)

For learning purpose, we would use InMemorySaver.

Don't forget to specify thread_id every time you invoke if you need short term memory to be enabled.
Making it remember things in a way you like could be done through custom state.

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver


agent = create_agent(
    "gpt-5",
    tools=[get_user_info],
    checkpointer=InMemorySaver(),
)

agent.invoke(
    {"messages": [{"role": "user", "content": "Hi! My name is Bob."}]},
    {"configurable": {"thread_id": "1"}},
)
```

1.3.1 Core Components of LangChain (without Middleware): Tools

You might have used or defined some tools previously (in QuickStart or you played on your own).

Let's make better tools!

Tool, in a glance, is a function that might or might not be called by LLM during conversation.

Context in ToolRuntime works like user structured input I would say.

You could make a schema for a tool to limit its range of acceptable inputs.

Schema for tool works like structured input, again 🤔.

What are the differences between context and state?

Advanced schema definition

Define complex inputs with Pydantic models or JSON schemas:

Pydantic model | JSON Schema

```python
from pydantic import BaseModel, Field
from typing import Literal

class WeatherInput(BaseModel):
    """Input for weather queries."""
    location: str = Field(description="City name or coordinates")
    units: Literal["celsius", "fahrenheit"] = Field(
        default="celsius",
        description="Temperature unit preference"
    )
    include_forecast: bool = Field(
        default=False,
        description="Include 5-day forecast"
    )

@tool(args_schema=WeatherInput)
def get_weather(location: str, units: str = "celsius", include_forecast: bool
    """Get current weather and optional forecast."""
    temp = 22 if units == "celsius" else 72
    result = f"Current weather in {location}: {temp} degrees {units[0].upper(
    if include_forecast:
        result += "\nNext 5 days: Sunny"
    return result
```

2. Hands-On Project

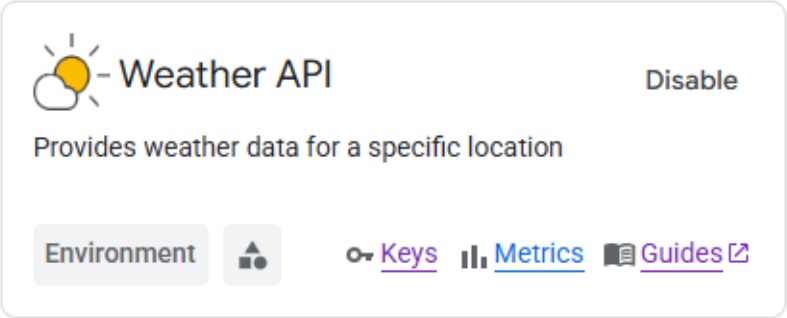
2. Hands-On Project: Travel Plan after Retirement

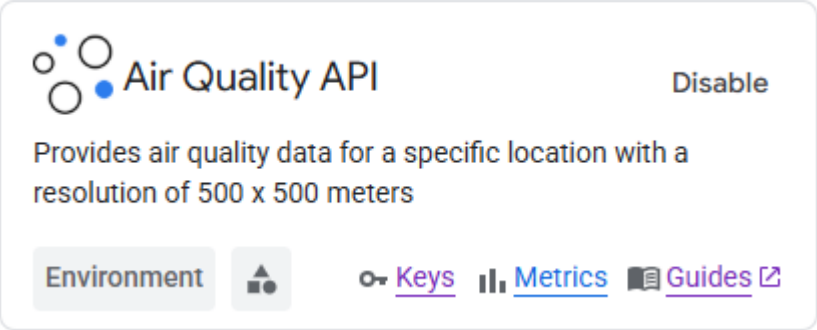
We would do a fun project with Google Maps API key!!

Project Description: Michael Zhang retired and he would like to travel around the world. He needs a Python program to do trips planning for him, which is what agent is good at accidentally. He also loves passing by multiple cities a time. For example, he might go to Toronto first and stay there for couple days, then go to New York, finally Miami.
He wants a program that could accept multiple city names and period of time of stay for each city and could output a detailed multi-city travel plan. Since LLM can't predict weather but Google Maps API could, you are expected to build single agent application using LangChain framework that could use tools to query weather and others.

Michael might have question for generated travel plan so short-term memory must be enabled otherwise Michael would be mad!

Michael would like to know what cloths to wear in each city and whether he should use umbrella so you must use google weather API to forecast weather!
He cares about air quality and would like to know how many masks he needs in total (if a city's air quality is bad, Michael would put on a mask. But what's bad is up to you when developing python program as long as you don't make it ridiculous).





Sobey School of Business
Saint Mary's University
MBAN

2. Hands-On Project: Travel Plan after Retirement

Don't worry too much about the billing, Google won't charge you for doing such a small project!!!! So don't be afraid of confirming you card information in Google's billing.
If you like you could make a Chat UI, that's better!
I personally don't need you to make slides to present your program to Michael, just

SHOW YOUR DEMO AND CODE!

Quick notes:
Michael won't go to North Korea. Just within North America is okay. Since only 4 days of forecast of air quality are available, you can assume Michael flies ultra fast and no time spent in traffic and Michael starts his travel from the first city he plans to visit. Pay attention to "currentTime" in the JSON object. You might need to use a model that could reason. If you need, configure thinking-budget in initialization of model. Please choose a LLM that could call tools. Last but not least, I might not be able to help you instantly so please debug on your own.

Forecast

Release Notes

Page Summary

The `forecast` endpoint provides up to 96 hours (4 days) of hourly air quality forecast information for a given location. includes hourly air quality indexes as well as additional information:

This is what I got from querying current weather of Mountain View, CA (at the time I was writing this slide)


```
{'currentTime': '2025-12-28T19:12:39.081836392Z', 'timeZone': {'id': 'America/Los_Angeles'}, 'isDaytime': True, 'weatherCondition': {'iconBaseUri': 'https://maps.gstatic.com/weather/v1/sunny', 'description': {'text':
  'Sunny', 'languageCode': 'en'}, 'type': 'CLEAR'}, 'temperature': {'degrees': 10.7, 'unit': 'CELSIUS'}, 'feelsLikeTemperature': {'degrees': 10.7, 'unit': 'CELSIUS'}, 'dewPoint': {'degrees': 2.3, 'unit': 'CELSIUS'},
 'heatIndex': {'degrees': 10.7, 'unit': 'CELSIUS'}, 'windChill': {'degrees': 10.7, 'unit': 'CELSIUS'}, 'relativeHumidity': 56, 'uvIndex': 2, 'precipitation': {'probability': {'percent': 0, 'type': 'RAIN'}, 'snowQpf':
  {'quantity': 0, 'unit': 'MILLIMETERS'}, 'qpf': {'quantity': 0, 'unit': 'MILLIMETERS'}}, 'thunderstormProbability': 0, 'airPressure': {'meanSeaLevelMillibars': 1026.7}, 'wind': {'direction': {'degrees': 315,
 'cardinal': 'NORTHWEST'}, 'speed': {'value': 2, 'unit': 'KILOMETERS_PER_HOUR'}, 'gust': {'value': 3, 'unit': 'KILOMETERS_PER_HOUR'}}, 'visibility': {'distance': 16, 'unit': 'KILOMETERS'}, 'cloudCover': 3,
 'currentConditionsHistory': {'temperatureChange': {'degrees': -1.1, 'unit': 'CELSIUS'}, 'maxTemperature': {'degrees': 13.3, 'unit': 'CELSIUS'}, 'minTemperature': {'degrees': 4, 'unit': 'CELSIUS'}, 'snowQpf':
 {'quantity': 0, 'unit': 'MILLIMETERS'}, 'qpf': {'quantity': 0, 'unit': 'MILLIMETERS'}}}
```


This is what I got from converting 1600 Amphitheatre Parkway, Mountain View, CA to its lat and lon.


```
C:\Users\s5615543\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\s5615543\PycharmProjects\PythonProject\google.py
{'results': [{'address_components': [{'long_name': 'Google Building 41', 'short_name': 'Google Building 41', 'types': ['premise']}, {'long_name': '1600', 'short_name': '1600', 'types': ['street_number']},
 {'long_name': 'Amphitheatre Parkway', 'short_name': 'Amphitheatre Pkwy', 'types': ['route']}, {'long_name': 'Mountain View', 'short_name': 'Mountain View', 'types': ['locality', 'political']}, {'long_name': 'Santa
 Clara County', 'short_name': 'Santa Clara County', 'types': ['administrative_area_level_2', 'political']}, {'long_name': 'California', 'short_name': 'CA', 'types': ['administrative_area_level_1', 'political']},
 {'long_name': 'United States', 'short_name': 'US', 'types': ['country', 'political']}, {'long_name': '94043', 'short_name': '94043', 'types': ['postal_code']}, {'long_name': '1351', 'short_name': '1351', 'types':
 ['postal_code_suffix']}], 'formatted_address': 'Google Building 41, 1600 Amphitheatre Pkwy, Mountain View, CA 94043, USA', 'geometry': {'location': {'lat': 37.43669209999999, 'lng': -122.0550336}, 'location_type':
 'ROOFTOP', 'viewport': {'northeast': {'lat': 37.4380410802915, 'lng': -122.0536846197085}, 'southwest': {'lat': 37.4353431197085, 'lng': -122.0563825802915}}}, 'place_id': 'ChIJxQvW8wK6j4AR3ukttGy3w2s', 'types':
 ['premise', 'street_address']}], 'status': 'OK'}
```

2. Hands-On Project: Easy & Hard

This project could be hard or easy:

Easy:

Your single agent program would take the following as input, literally, and agent would be informed by messages of tools you defined that call Google Maps APIs to retrieve forecast of Weather and Air Quality, then output a travel report.

City1: Toronto 2025-01-31
CN Tower;290 Bremner Blvd, Toronto, ON M5V 3L9;8am-9am
Royal Ontario Museum;100 Queens Park, Toronto, ON M5S 2C6;10am-11am
City2: Chicago 2025-02-01
The Art Institute of Chicago;111 S Michigan Ave, Chicago, IL 60603, United States;9am-11am
Griffin Museum of Science and Industry;5700 S DuSable Lake Shore Dr, Chicago, IL 60637, United States;12pm-13pm

2. Hands-On Project: Easy & Hard

Hard:

Your single agent program would take the following as input, literally:

City1: Toronto 2025-01-31
CN Tower;8am-9am
Royal Ontario Museum;10am-11am
City2: Chicago 2025-02-01
The Art Institute of Chicago;9am-11am
Griffin Museum of Science and Industry;12pm-13pm

(A tool that can generate names of some tourist attractions given a city name is needed. You might need to use structured output to get a list of names of tourist attractions. To retrieve the geographical address, like 923 Robie St, Halifax, NS B3H 1G3, of a place, you need to use Text Search API: https://developers.google.com/maps/documentation/places/web-service/text-search
I provided the code snippet and docs to help you succeed.)

```python
import os
import requests
from dotenv import load_dotenv

load_dotenv()
GOOGLE_MAPS_API_KEY = os.environ["GOOGLE_MAPS_API_KEY"]
url = "https://places.googleapis.com/v1/places:searchText"
try:
    headers = {
        "Content-Type": "application/json",
        "X-Goog-Api-Key": GOOGLE_MAPS_API_KEY,
        "X-Goog-FieldMask": "places.formattedAddress",
    }
    payload = {"textQuery": "CN Tower"}
    response = requests.post(url, headers= headers, json = payload)  # or get, it depends
    response.raise_for_status()  # Raise an exception for bad status codes (4xx or 5xx)
    data = response.json()
    print(data)
except requests.exceptions.RequestException as e:
    print(f"Error fetching weather data: {e}")
```

google

```
C:\Users\[redacted]\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\[redacted]\PycharmProjects\PythonProject\google.py
{'places': [{'formattedAddress': '290 Bremner Blvd, Toronto, ON M5V 3L9, Canada'}]}
```

Home > Products > Google Maps Platform > Documentation > Web Services > Places API > Guides

Was this helpful?

Text Search (New)

Send feedback

Release Notes

Page Summary

Select platform: Android iOS JavaScript Web Service

European Economic Area (EEA) developers

Introduction

Text Search (New) returns information about a set of places based on a string (for example, "pizza in New York" or "shoe stores near Ottawa" or "123 Main Street"). The service responds with a list of places matching the text string and any location bias that has been set.

In addition to required parameters, Text Search (New) supports refining queries using optional parameters for better results.

The APIs Explorer lets you make live requests so that you can get familiar with the API and the API options:

2. Hands-On Project: Easy & Hard

We want you to do the hard one. Btw there are 4 projects you need to do in total for this special journey.

If you could finish this project in hard mode, your grading scheme would become 18 marks for this project and 4 marks for each of upcoming projects.

Otherwise, each counts 7.5 mark.

If you would like to do more, you could even make a startup based on this idea.

Thanks for your hard work.

*Possibly helpful starter 🤔

(Using .env file is always a good practice and I suggest you do it all the time…..)
(Simply copy and paste won't work at all scenarios. Adapt the following code whenever needed. The following code only serves as suggestion, and we are not responsible for any consequence that results in being billed by Google. The APIs we suggested to use, Air Quality, Weather, Text Search won't charge any money.)

```python
import os

import requests
from dotenv import load_dotenv

load_dotenv()
GOOGLE_MAPS_API_KEY = os.environ["GOOGLE_MAPS_API_KEY"]
# Example URL for current conditions
#url = f"https://weather.googleapis.com/v1/currentConditions:lookup?key={GOOGLE_MAPS_API_KEY}&location.latitude=37.4220&location.longitude=-122.0841"
#url = f"https://maps.googleapis.com/maps/api/geocode/json?address=1600+Amphitheatre+Parkway,+Mountain+View,+CA&key={GOOGLE_MAPS_API_KEY}"
url = f"https://airquality.googleapis.com/v1/forecast:lookup?key={GOOGLE_MAPS_API_KEY}"
try:
    response = requests.post(url) #or get, it depends
    response.raise_for_status() # Raise an exception for bad status codes (4xx or 5xx)
    weather_data = response.json()
    print(weather_data)
except requests.exceptions.RequestException as e:
    print(f"Error fetching weather data: {e}")
```

Thank you for your paged attention!

arXiv > cs > arXiv:2309.06180

Computer Science > Machine Learning

[Submitted on 12 Sep 2023]

Efficient Memory Management for Large Language Model Serving with PagedAttention

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica